

1 Methodology

Relevant marks: Model adaptation and RAG methodology (20 marks).

1.1 Baseline System

The `baseline`, `PromptOnlyBaseline` (`src/baseline.py`), returns a fixed, generic answer drawn from a small lookup of play-level priors (e.g., a one-sentence summary of *Macbeth*). It does *not* call the retriever and therefore cannot cite a scene, act, or speaker. The baseline is intentionally weak: its purpose is to expose the contribution of retrieval rather than to compete on absolute quality. A second variant, `KeywordRetrievalBaseline`, uses Jaccard overlap over scene summaries; it is included in the repository for completeness but is not the primary baseline reported here. We chose the prompt-only variant as the primary baseline because the assignment specification explicitly lists “prompt-only generation” as an acceptable baseline.

1.2 RAG-Based System

The RAG system (`src/rag_chatbot.py`) has five components.

Embedder / retriever. Implemented in `src/retrieval.py`. The class `EmbeddingRetriever` has a single public API (`build_index`, `retrieve`) and two interchangeable backends: (a) `sentence-transformers/all-MiniLM-L6-v2` dense embeddings (22M parameters, 384-d, CPU-friendly), used when the package is installed; (b) a scikit-learn TF-IDF backend used as a fallback. The active backend is exposed via `retriever.backend_name` and recorded in `results/evaluation_summary.json`. **All evaluation numbers reported here were produced with the MiniLM backend.**

Top- k . We use $k = 3$. With 73 scene chunks, this returns roughly one scene per play on average; a beginner-friendly answer typically needs only the highest-scoring scene as evidence.

Prompt structure. `src/rag_chatbot.py` builds a prompt that injects the retrieved evidence between a system prompt (`prompts/system_prompt.txt`, instructing the model to use only retrieved context, indicate uncertainty, and avoid invented details) and the user query. The final prompt is the concatenation of the system prompt, the top- k retrieved chunks (each annotated with “[Context i | similarity= s]” headers), and the user query under an “Answer:” label.

Generator: phi3:3.8b via Ollama (default). The default generator is Microsoft’s `phi3:3.8b` (3.8B parameters, instruction-tuned, 4-bit quantised) called over the local Ollama HTTP API at `http://localhost:11434`. The model receives the system prompt, the top- k retrieved scenes, and the user question (see “Prompt structure” above) and returns a beginner-friendly answer constrained by the prompt. We picked `phi3:3.8b` because it is one of the smallest competent instruction-tuned models that runs on a

standard laptop without a GPU, satisfies the assignment’s “small, deployable, resource-constrained” framing, and is fully local (no data leaves the host).

Fallback generator: extractive composer. If Ollama is not running, the system silently falls back to a deterministic extractive composer (`generation.ExtractiveComposer`) that constructs each answer from the retrieved scene summaries, keywords, and one verbatim quote. The fallback guarantees the system still works on machines without Ollama installed.

Stylised composer. `generation.StylisedComposer` produces a ≤ 150 -word Shakespearean-style verse, prefixed with “[STYLISTED CREATIVE OUTPUT – not textual evidence; based on the retrieved scenes.]” as required by the specification.

1.3 Model Choice and Justification

We deliberately avoided an external hosted API and a large local generative model. The assignment’s pedagogical focus is on “*purpose-built systems that can operate under constraints involving cost, privacy, latency, maintainability, and domain specificity*”. `phi3:3.8b` satisfies all five constraints:

- **Cost:** zero inference cost; runs entirely on the student laptop after a one-time ~ 2 GB model download.
- **Privacy:** fully local; no data leaves the host.
- **Latency:** 20–60 seconds per CPU answer; fast enough for an interactive demo.
- **Maintainability:** one HTTP endpoint, one model file, no Docker, no GPU drivers.
- **Domain specificity:** retrieval-first prompting binds the model’s output to the Shakespeare corpus rather than its general prior.

The retrieval backend (MiniLM) is similarly small: 22M parameters, 384-d embeddings, a few hundred kilobytes of weights, and CPU inference. The whole stack fits comfortably under the assignment’s “runs on a standard personal computer” constraint.

2 Results (model-side excerpt)

The full evaluation harness is owned by the Evaluation Lead. The table below reports the headline numbers attributable to the model choice. Scores are means across 12 evaluation questions on the 1–5 rubric defined in `src/evaluate.py`.

Interpretation. Retrieval gains (+3.83) come from MiniLM, which retrieves the on-topic scene at average cosine similarity 0.45–0.72. Correctness gains (+1.60) come from `phi3`: it can fuse the retrieved scene summaries into a coherent beginner-friendly answer that a fixed-template generator cannot produce. Grounding (+0.67) and style (0.00) are weaker – `phi3` paraphrases evidence rather than emitting verbatim “Act i , Scene j ” citations, and its stylised verse is

Table 1: Mean evaluation scores across 12 questions, 1–5 scale (higher is better).

System	Corr.	Ground	Retr.	Useful	Style
Baseline	2.60	2.33	1.00	3.00	3.00
RAG (phi3 + MiniLM)	4.20	3.00	4.83	3.17	3.00
Δ	+1.60	+0.67	+3.83	+0.17	0.00

fluent but does not always include the discrete archaic markers our rubric counts. A stricter system prompt could close either gap; we deliberately did *not* apply that constraint here so the report reflects the unconstrained behaviour of phi3:3.8b.

3 Representative Prompts

System prompt (prompts/system_prompt.txt):

You are a Shakespeare-aware assistant.
 Use the retrieved context to answer the user's question.
 Your answer must be beginner-friendly.
 If the retrieved context is insufficient, say so clearly.
 Do not invent unsupported details.
 If asked to produce a Shakespearean-style response, keep it under 150 words and make clear that it is creative output, not textual evidence.

Stylised prompt (prompts/stylised_prompt.txt):

You are a Shakespeare-aware assistant producing a short stylised response.
 Rules:
 - Maximum 150 words.
 - Adopt early-modern English register ("thou", "thy", "ere", "o'er", "doth").
 - Creative output; must be labelled as such.
 - Remain interpretable to a beginner.
 - Use the retrieved scenes only as inspiration for themes (ambition, love, prophecy, etc.); do not quote them as the user's question.

4 GenAI Usage (model-side rows)

Table 2: Model Lead’s contribution to the group’s GenAI usage log.

Tool	Prompt or Task	Useful output	Verification / Modification
ChatGPT Claude	/ “Write a small wrapper around the Ollama HTTP API that exposes a single <code>generate(prompt)</code> method.”	Suggested a 70-line <code>OllamaGenerator</code> class with an <code>available()</code> probe and a 120s request timeout.	Adopted, after testing end-to-end against <code>phi3:3.8b</code> . We added the silent fall-back to the extractive composer ourselves.
ChatGPT Claude	/ “Draft a Shakespeare-aware system prompt that keeps the model grounded in retrieved context.”	Returned a 6-line prompt emphasising beginner-friendliness, refusal on insufficient evidence, and a 150-word cap on stylised output.	Accepted as v1. Tested by running <code>python evaluate.py</code> on all 12 questions; results in Sec. 2. A stricter citation-forcing variant was considered and discussed in the failure analysis.
ChatGPT Claude	/ “Suggest archaic register substitutions for a stylised generator (you $\rightarrow$ thou, ...).”	Provided pairs incl. <code>before$\rightarrow$ere</code> , <code>over$\rightarrow$o’er</code> , <code>often$\rightarrow$oft</code> .	Cherry-picked the conservative substitutions that round-trip safely; discarded those that produced ungrammatical output.